

✓ ITAI 1378 - FINAL PROJECT SUBMISSION

Due: December 4 (Week 15) - Present in class

Worth: 35% of your final grade

OBJECTIVE

Deliver a **working computer vision system** with:

-  Complete, documented code
-  Live demonstration
-  Performance evaluation
-  Video demo

No long written report! Just code, demo, and presentation.

WHAT TO SUBMIT

1. Complete GitHub Repository

All your code, documentation, and results

2. Demo Video (3-5 minutes)

Screen recording showing your system working

3. Final Presentation (10 minutes)

In-class presentation with **live demo**

GITHUB REPOSITORY STRUCTURE

Your final repository should look like this:

```
project-name/
├── README.md           ← Comprehensive project documentation
├── requirements.txt    ← All Python dependencies
├── environment.yml     ← (Optional) Conda environment
├──
├── data/
│   ├── raw/           ← Original data (or download link)
│   └── processed/      ← Preprocessed data
```

```

|   └─ sample/                ← Sample images for quick demo
|   └─ README.md              ← Dataset documentation
|
|   └─ models/
|       └─ pretrained/        ← Downloaded models (or links)
|       └─ trained/           ← Your trained models (or links)
|       └─ README.md          ← Model documentation
|
|   └─ notebooks/
|       └─ 01_data_exploration.ipynb    ← Explore dataset
|       └─ 02_model_training.ipynb      ← Train/fine-tune
|       └─ 03_evaluation.ipynb          ← Test & evaluate
|       └─ 04_demo.ipynb                ← Interactive demo
|
|   └─ src/                        ← (Optional) Python scripts
|       └─ data_processing.py
|       └─ model.py
|       └─ train.py
|       └─ inference.py
|       └─ utils.py
|
|   └─ results/
|       └─ images/                ← Output images with predictions
|       └─ metrics.txt            ← Performance metrics
|       └─ visualizations/        ← Charts, confusion matrices
|
|   └─ docs/
|       └─ proposal.pdf           ← Your midterm proposal
|       └─ presentation.pdf       ← Final presentation slides
|       └─ AI_usage_log.md        ← AI tool documentation
|
|   └─ demo_video.mp4             ← Or link to YouTube

```

README.md (Final Version)

Your README is the **most important file**. Use this template:

```
# [Your Project Name]
```

```
![Demo Screenshot or GIF](results/demo.gif)
```

```
*One-sentence description of what your project does*
```

```

## Team Members
- [Name 1] - [Role/Contribution]
- [Name 2] - [Role/Contribution]
- [Name 3] - [Role/Contribution]

## Project Tier
**Tier [1/2/3]** - [Why this tier? What makes it this level?]

---

## 🎯 Problem & Solution

### The Problem
[2-3 sentences: What real-world problem does this solve?]

### Our Solution
[2-3 sentences: How does your system solve it?]

### Impact
[Who benefits? How much time/money saved? What's the value?]

---

## 🛠️ Technical Details

### Approach
- **Task**: Object Detection / Classification / Segmentation / VLM / etc.
- **Model**: YOLOv8 / ResNet / CLIP / etc.
- **Framework**: PyTorch / TensorFlow / Hugging Face
- **Key Libraries**: ultralytics, opencv-python, etc.

### System Architecture
[Include your system diagram - image or flowchart]

```

[Input] → [Preprocessing] → [Model] → [Post-processing] → [Output]

Dataset

- Source: [Where you got the data - link if public]
- Size: [Total images/videos]
- Classes: [List of classes/categories]
- Split: Train: X images, Val: Y images, Test: Z images

- Preprocessing: [What you did: resizing, augmentation, normalization]



How to Run Installation bash

```
# Clone repository
git clone https://github.com/username/project-name.git
cd project-name

# Install dependencies
pip install -r requirements.txt
```

✓ Quick Start

```
# Option 1: Run Jupyter notebook
jupyter notebook notebooks/04_demo.ipynb

# Option 2: Run Python script
python src/inference.py --image data/sample/test.jpg
```

✓ Detailed Usage

Step 1: Prepare your data

```
python src/data_processing.py --input data/raw --output data/processed
```

Step 2: Run inference

```
python src/inference.py --image path/to/image.jpg --output results/
```

Step 3: View results Results saved to `results/` folder



Results

Performance Metrics

Metric	Value
Accuracy	92.5%
Precision	90.1%
Recall	89.3%

Metric	Value
F1-Score	89.7%
Inference Time	0.05s per image
FPS (video)	20 FPS

Visualizations

Success Cases:

![Success Example 1] (results/images/success_1.jpg)

![Success Example 2] (results/images/success_2.jpg)

Failure Cases:

![Failure Example] (results/images/failure_1.jpg)

Why it failed: [Explain the failure - poor lighting, occlusion, etc.]

Comparison with Baseline

Approach	Accuracy	Speed
Manual Process	N/A	2 hours
Simple Threshold	65%	0.01s
Our System	92.5%	0.05s

Demo Video

 [Watch Demo on YouTube](#)

3-5 minute video showing the system in action

Key Learnings

What Worked Well

- [Success 1]
- [Success 2]
- [Success 3]

Challenges Faced

Challenge 1: [Description]

- **Solution:** [How you solved it]

Challenge 2: [Description]

- **Solution:** [How you solved it]

What We'd Do Differently

- [Improvement 1]
 - [Improvement 2]
-



AI Usage Documentation

See detailed log: [docs/AI_usage_log.md](#)

Summary:

- **Used AI for:** Understanding architectures, debugging, code generation
 - **Key learnings:** [What you learned from AI]
 - **Code attribution:** ~70% written by us, ~30% AI-assisted
-



Future Improvements

- ☐ [Improvement 1 - be specific]
 - ☐ [Improvement 2 - be specific]
 - ☐ [Improvement 3 - be specific]
-



References

1. [Paper/Article 1]
 2. [Dataset source with link]
 3. [Model documentation]
 4. [Tutorial followed]
-



License

[MIT / Apache / Academic Use Only]



Acknowledgments

- Thanks to Professor [Name] for guidance
- Pre-trained models from [Ultralytics / Hugging Face]
- Dataset from [Source]
- AI assistance from [ChatGPT / Claude / Copilot]

DEMO VIDEO REQUIREMENTS (3-5 minutes)

Part 1: Introduction (30 seconds)

"Hi, I'm [name], and this is [project name].
It solves [problem] by [solution]."

Part 2: Live Demo (2-3 minutes)

Show your system actually working!

- Run it on 3-5 different test cases
- Show both **successes AND failures**
- Explain what's happening step by step

Example narration:

"Here's an image of a warehouse shelf. When I run detection... [run code]... you can see it correctly identified 15 products in 0.05 seconds.

Now a harder case with poor lighting... [run code]... It detected 12 out of 13 products - pretty good!

Here's a failure case... [run code]... It missed this product because it's occluded. Future work!"

Part 3: Results (1 minute)

- Show metrics (accuracy, speed)
- Show visualizations (confusion matrix, graphs)
- Compare with baseline

Part 4: Conclusion (30 seconds)

- What you learned
- Future improvements
- Thank you!

Recording Tips:

- Use Zoom, OBS, or QuickTime to record screen
- Show your face (optional but nice!)
- Practice once before recording
- Authenticity > Perfection!

Upload Options:

- YouTube (unlisted link)
- Google Drive (shareable link)
- Include link in README



FINAL PRESENTATION (10 minutes + 5 min Q&A)

Slide Structure (10-12 slides):

1. **Title** - Project name, team, tier
2. **Problem & Motivation** - What problem, why it matters
3. **Solution Overview** - What you built, how it works
4. **Technical Approach (2 slides)** - CV technique, model, architecture diagram
5. **Dataset & Preprocessing** - Data source, size, preprocessing steps
6. **LIVE DEMO (5 minutes)** - Run your system live in front of class! Most important part!
7. **Results (2 slides)** - Metrics, visualizations, success/failure cases
8. **Key Learnings** - What worked, what didn't, challenges overcome
9. **Future Work** - How you'd improve it
10. **Acknowledgments** - Thank professor, teammates, AI tools

Live Demo Requirements:

- **Must run live during presentation**
- Show 2-3 test cases
- Explain what's happening
- Have backup video ready (just in case!)



GRADING RUBRIC (100 points)

Component	Points	Criteria
System Works	35	Does it work? Is it reliable? Can you demo it live?
Code Quality	20	Well-organized, documented, reproducible, runs without errors
Results	15	Good metrics, thorough evaluation, honest analysis

Component	Points	Criteria
GitHub Repository	10	Complete structure, excellent README, easy to understand
Demo Video	10	Clear demonstration, good explanation, shows system working
Presentation	10	Clear slides, successful live demo, answers questions well

Total: 100 points

Bonus Points Available:

- **Tier 2 Project:** +10 points
- **Tier 3 Project:** +20 points
- **Exceptional Documentation:** +5 points
- **Creative/Novel Solution:** +5 points
- **Helping Classmates:** +5 points

Maximum Possible: 145 points

PRE-SUBMISSION CHECKLIST

Week Before (Nov 27 - Dec 3):

- ☐ All code works and is tested
- ☐ README.md is complete and detailed
- ☐ Demo video is recorded and uploaded
- ☐ Presentation slides are ready
- ☐ Live demo is tested and working
- ☐ AI usage log is complete
- ☐ All files are in GitHub repository
- ☐ Repository is clean (no unnecessary files)
- ☐ requirements.txt includes all dependencies
- ☐ Sample data is included for quick testing

Day of Presentation (Dec 4):

- ☐ Laptop/device is charged
 - ☐ Demo environment is tested
 - ☐ Backup video is ready (USB drive or cloud)
 - ☐ Presentation is loaded and ready
 - ☐ GitHub link is ready to share
 - ☐ Can answer questions about your code
 - ☐ Team members know their parts
-



TIPS FOR SUCCESS

1. **Test everything multiple times** - Murphy's Law applies to demos!
 2. **Have a backup** - Video, screenshots, or alternative demo ready
 3. **Practice your live demo** - Know exactly what to click and when
 4. **Be honest about failures** - Good analysis of poor results is valuable
 5. **Explain your thinking** - Show you understand the code
 6. **Time management** - Don't spend 15 minutes on intro, save time for demo!
 7. **Make it visual** - Show results, not just code
 8. **Tell a story** - Problem → Solution → Results → Future
-



COMMON QUESTIONS

Q: What if my demo fails during presentation?

A: Have your backup video ready! Explain what went wrong. We understand tech issues.

Q: Do I need perfect accuracy to get a good grade?

A: No! Good analysis of 70% accuracy > no analysis of 95% accuracy.

Q: Can I use code from the internet?

A: Yes, with attribution! But you must understand it and explain it.

Q: What if I'm not done by Dec 4?

A: Submit what you have! Partial credit for partial work. Communicate with instructor early.

Q: How much should I explain AI usage?

A: Be honest and detailed in your AI log. Show you used AI to learn, not to avoid learning.

Questions? Ask instructor BEFORE the deadline!

You've got this! 🚀

Last Updated: October 15, 2025